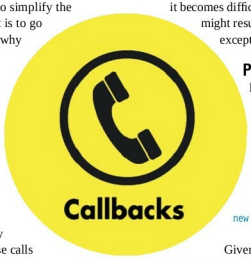
JavaScript: The New Parts

This eighth article in our series on JavaScript takes us to a feature known as Promises. This feature can be used in place of callbacks and to avoid its side effects. Promises are called futures or deferreds in other languages.

Before we get started, a word of caution. Understanding Promises could be as difficult as making sense of the Christopher Nolan movie 'Inception'. This article is an attempt to simplify the topic as much as possible. The attempt is to go beyond syntax and show examples on why Promises are indeed needed.

Promises are required if your application performs operations such as database access, file access or fetching user data from the online APIs of Twitter. These operations could take variable amounts of time. Waiting for these calls to complete could stall the application for a few seconds or freeze the UI. The result of the operation could even be an error, after a long wait and another retry attempt. Hence, it is better to make these calls asynchronous (not wait for the results).



JavaScript callbacks

JavaScript began as a browser language, with the primary task of handling user events such as a mouse click. The language has excellent support for event-driven programming. In this language, functions are first class citizens, which means you can use a function where any other primitive data type fits. For example, a function can be passed as an argument. A function can also return a function. A callback is a function that will get executed upon an event. It is common for frontend developers to write many callback functions on mouse events like onClick, onmousedown, etc. The functions (callback) get called upon mouse events.

Another important point to note is that JavaScript is single-threaded. Any operation (function call), which takes time, should be asynchronous.

Let's take an example of a callback.

```
1. var fs = require('fs');
2. fs.readFile('readme.txt', printfile);
3. function printfile(err, filecontents) {
4.    console.log(filecontents.toString());
5. }
```

printfile() is a callback function, which would be called after a read operation. One drawback of a callback function is that the flow of execution is different from the sequence of instructions written from top to bottom.

Another drawback of the callback function is called Pyramid of Doom. As we go on nesting multiple callbacks, it becomes difficult to debug. Some nested callbacks might result in an exception, and tracing exceptions is a nightmare.

Promise

Promise is a place holder for a future value. This is because we want to store the results of an asynchronous operation. Promise is an object. The syntax below is used to create a Promise.

The syntax is:

```
new Promise (function)
```

Given below is an example of how to create a function that returns a Promise. We have converted an asynchronous *fs.readFile()* operation into a Promise.

```
1. function readFile(filename) {
2.   let p = new Promise(function(resolve, reject) {
3.           fs.readFile(filename, function(err, contents) {
4.           if (err) {
5.              reject(err); // error case
6.           }
7.           resolve(contents);
8.         });
9.       });
10.     return p;
11. }
```

Line 2: Creates a new Promise object. The object 'p' gets a value after reading the file.

Line 3: A Promise takes a function. The function in turn takes two functions 'resolve' and 'reject' as parameters. For all successful cases, it executes 'resolve'. For error scenarios, the second parameter function is executed.

Promise states

Promise has states. Initially, when a Promise is created, it is in the pending state. This is the initial state. Once the asynchronous operation completes, the Promise moves to the resolved state. There are two possible states within 'resolve'. These are: 'fulfilled' and 'rejected'. A successful result of a